Group Project: Database For the Construction Submission Portal

Alexander Beck
CS BS/MS

Tina Cheng
CS Bachelor

Tran Man Tuyen Dinh
Master of CS

Zongyu Mu
Master of CS

Abstract

We are developing a database-driven web application using FastAPI and PostgreSQL.

We implemented database schema, backend setup, and API routing.

Goal: fully functional system with normalized design.

Project Overview

What is this project?

A fully functional RDBMS Database Application that powers a real-world construction permit management platform.

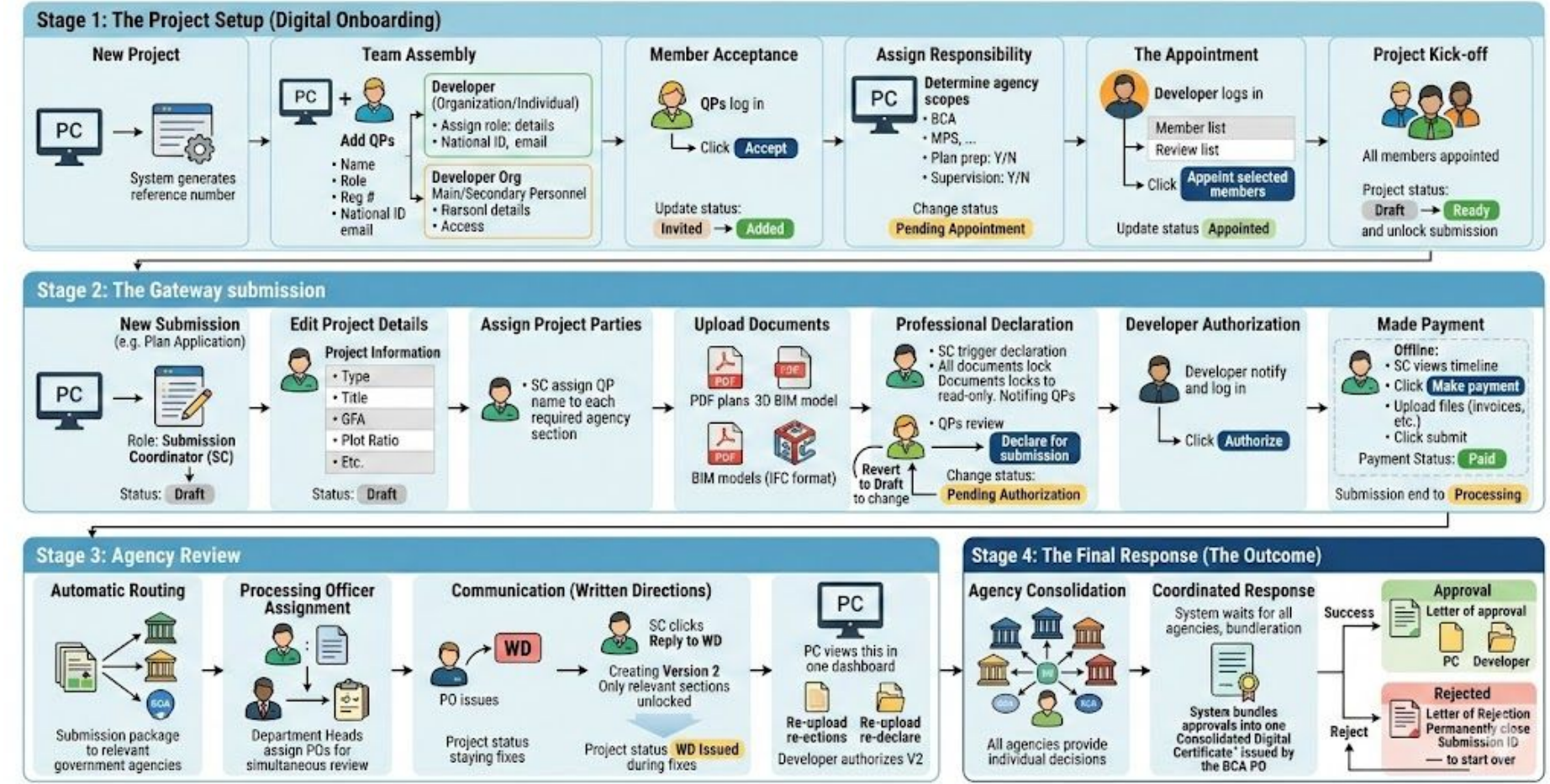
Design consultants can securely submit building plans - including 3D models and technical documents - to a central hub. Multiple agency bodies review, provide feedback, and approve submissions in parallel.

23.8 %
of new home price is regulatory cost
(NAHB, 2021)

14-15%
productivity gain from digitization
(McKinsey, 2016)

#1
among least digitized industries
(McKinsey, 2017)

A Comprehensive workflow visualization infographics the entire multi-stage process for Project Setup and Gateway Submission



User Requirements

Project Coordinator: Creates projects, assembles teams, manages multiple projects simultaneously. Recorded: Name, Role, Reg. No., National ID, Email.

Qualified Person: Licensed professionals with multiple scope assignments. Can also serve as Submission Coordinator (SC).

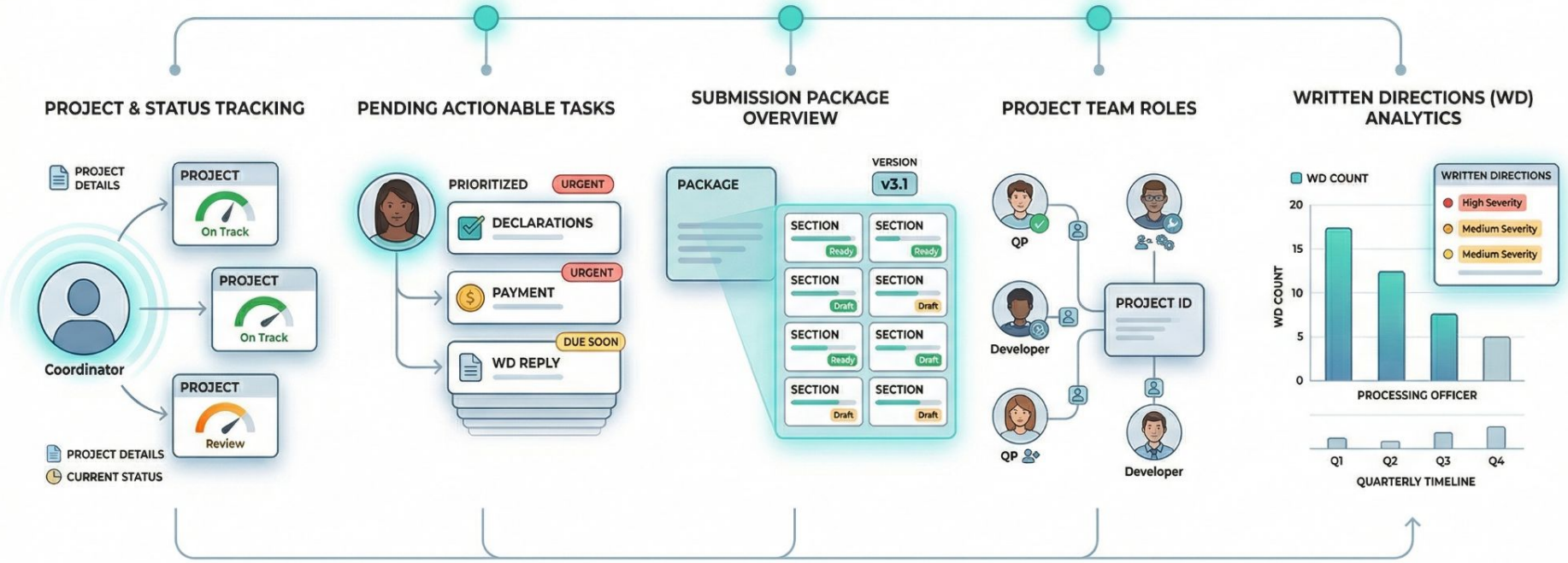
Developer: Client entity (Individual or Organization) that authorizes the project team and submission packages.

Agency & Processing Officer: Reviews submissions, issues Written Directions (WD), assigns POs by expertise and project type.

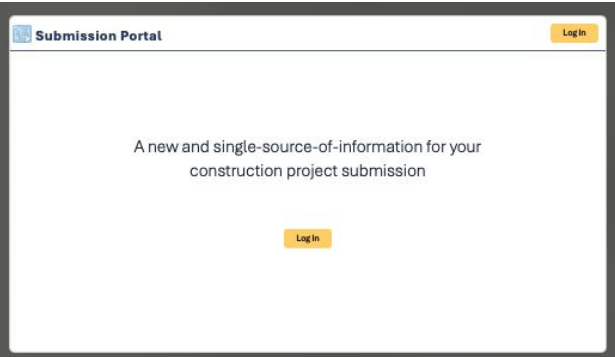
Submission Package: Multi-versioned application instance with statuses: Draft → Pending → Submitted → Processing → Approved/Rejected.

Documents & WD: Technical files (PDF, 3D models) tracked per section with read-only lock flags. WD feedback linked to sections.

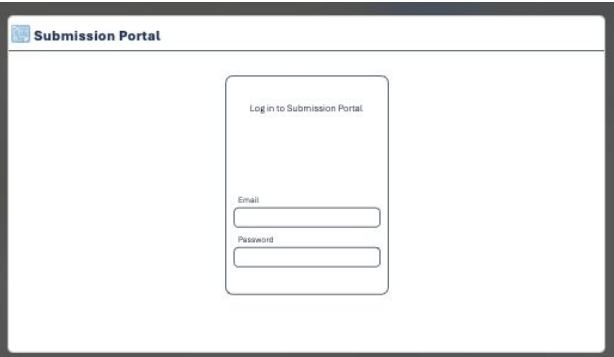
User Queries/Questions



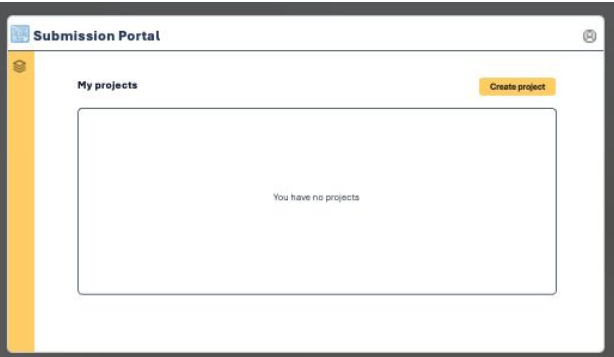
Users' Interface



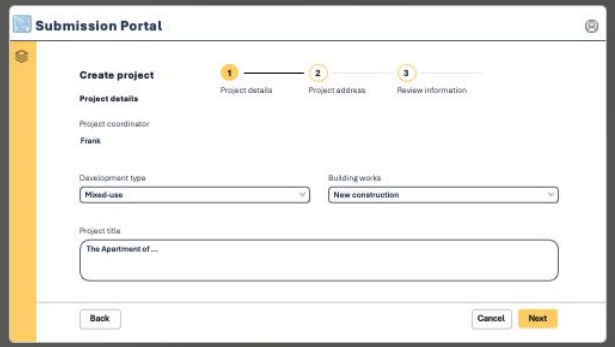
1



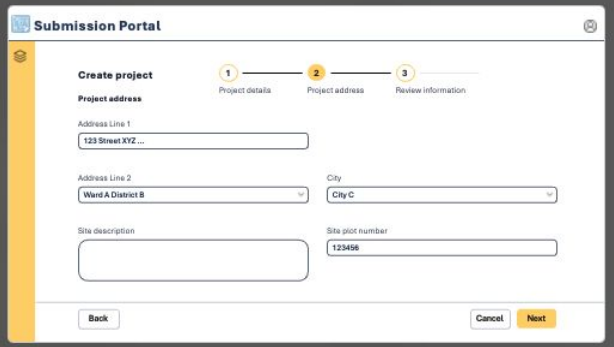
2



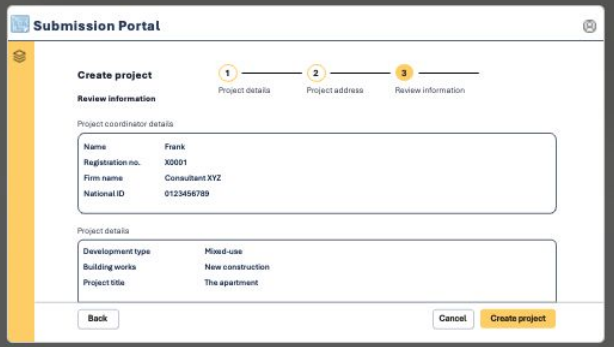
3



4



5



6

Tech Stack & System Design

Frontend

- HTML + CSS
- Tailwind CSS
- Login Page
- Project Table View
- File Upload Button

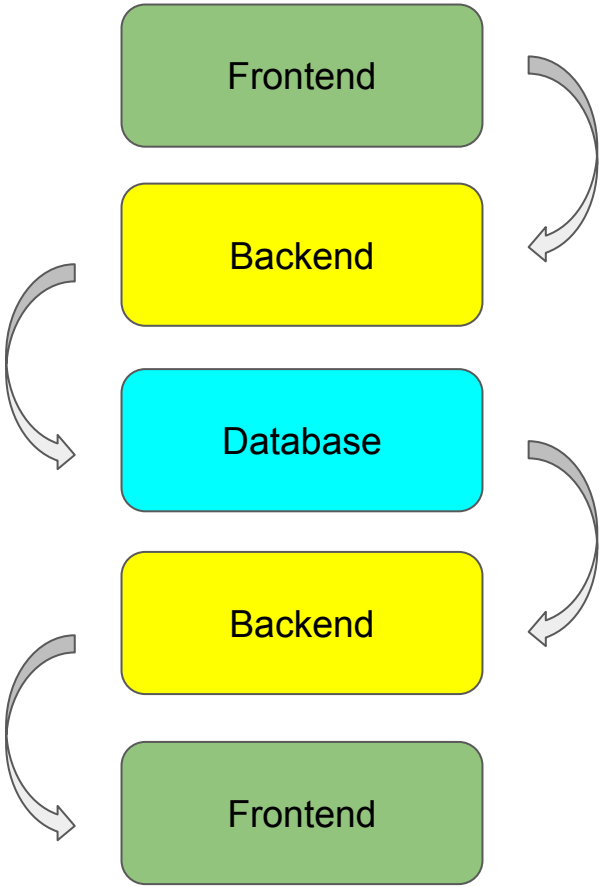
Backend

- Python + FastAPI
- SQLModel (ORM)
- JWT Authentication
- File Upload/Download API
- REST Endpoints

Database

- PostgreSQL
- 20+ Relational Tables
- FK / UNIQUE / CHECK
- Triggers & Constraints
- Normalization

Data Flow



Database Design

User: nationalID, name, email, status

Developer: IsA User (Individual or Organization)

ProjectCoordinator: IsA User + professionalNum, firmName

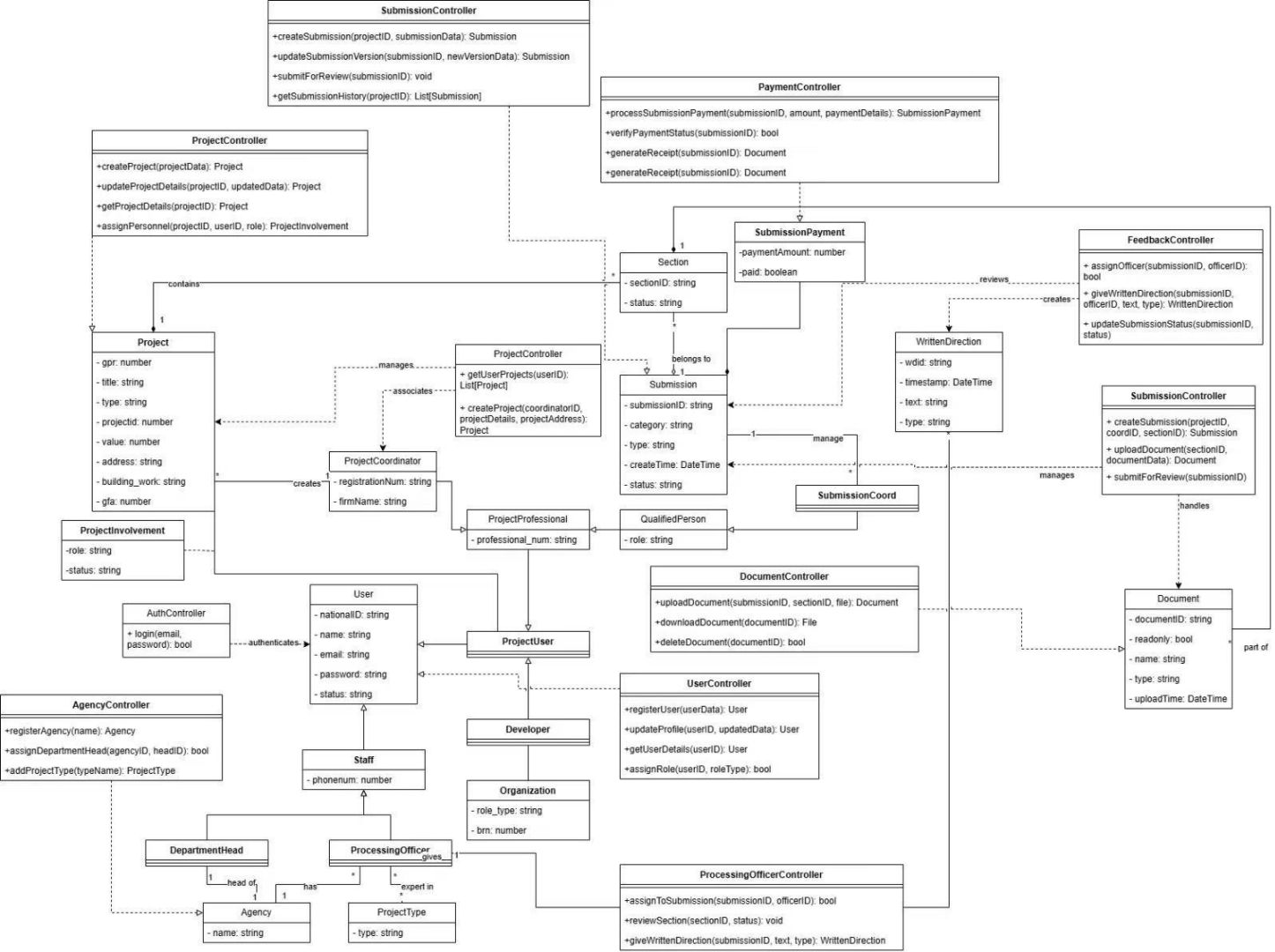
Project: projectID, title, address, buildingWork, gfa

QualifiedPerson: IsA User + professional_num, role

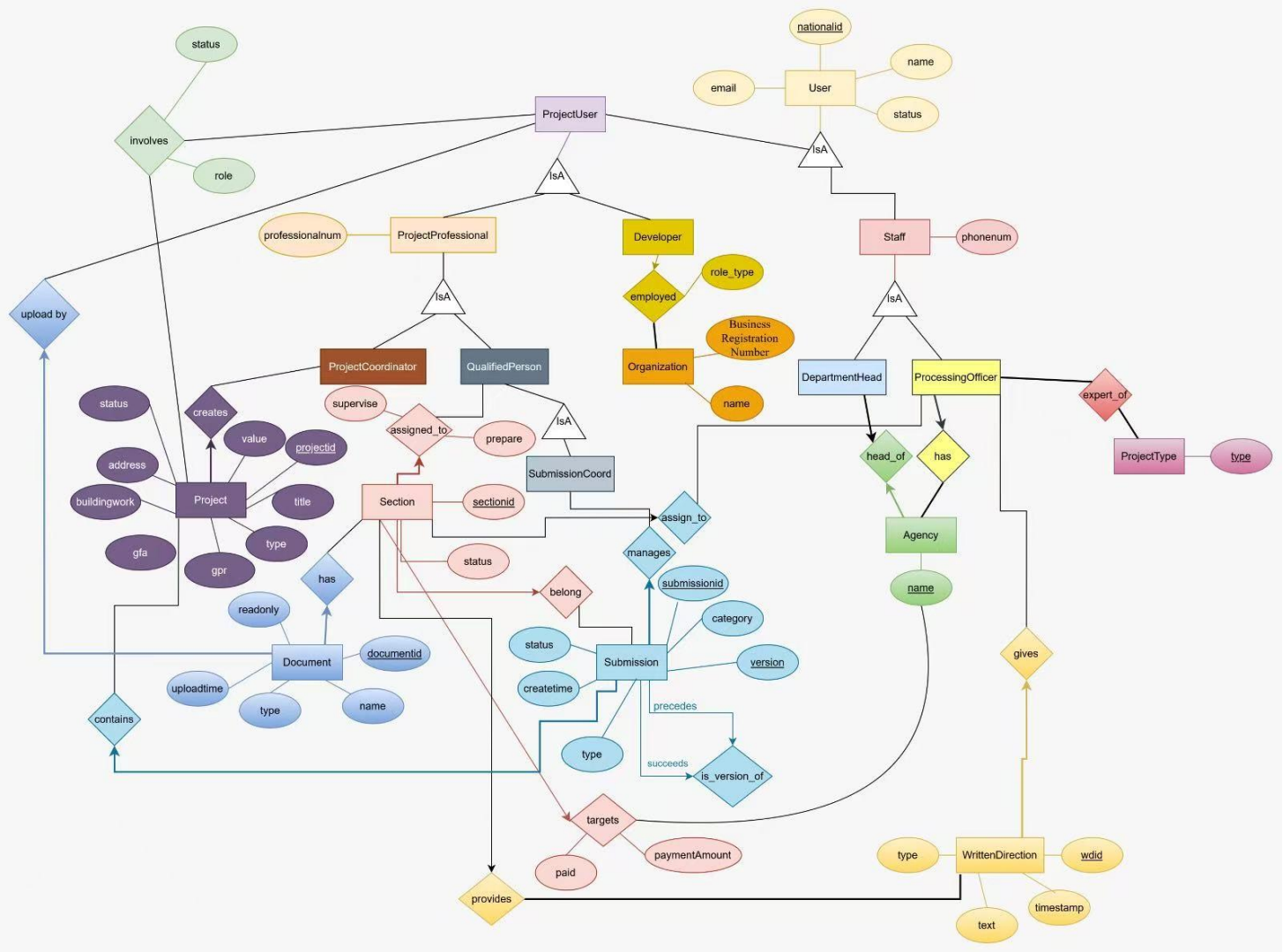
Submission: submissionID, category, type, version, status

Section: sectionD, agency, status, version

Document: documentID, name, type, readOnly, uploadTime

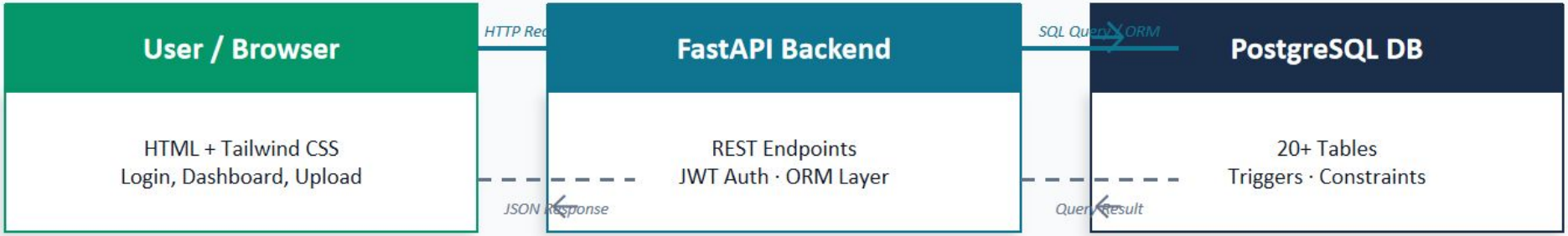

SubmissionController
+createSubmission(projectID, submissionData): Submission
+updateSubmissionVersion(submissionID, newVersionData): Submission
+submitForReview(submissionID): void
+getSubmissionHistory(projectID): List[Submission]
PaymentController
+processSubmissionPayment(submissionID, amount, paymentDetails): SubmissionPayment
+verifyPaymentStatus(submissionID): bool
+generateReceipt(submissionID): Document
+generateReceipt(submissionID): Document
ProjectController
+createProject(projectData): Project
+updateProjectDetails(projectID, updatedData): Project
+getProjectDetails(projectID): Project
+assignPersonnel(projectID, userID, role): ProjectInvolvement
SubmissionPayment
-paymentAmount: number
-paid: boolean
FeedbackController
+ assignOfficer(submissionID, officerID): bool
+ giveWrittenDirection(submissionID, officerID, text, type): WrittenDirection
+ updateSubmissionStatus(submissionID, status)
Section
- sectionID: string
- status: string
contains
reviews
creates
Project
- gpr: number
- title: string
- type: string
- projectId: number
- value: number
- address: string
- building_work: string
- gfa: number
manages
associates
ProjectController
+ getUserProjects(userID): List[Project]
+ createProject(coordinatorID, projectDetails, projectAddress): Project
belongs to
Submission
- submissionID: string
- category: string
- type: string
- createTime: DateTime
- status: string
WrittenDirection
- wdid: string
- timestamp: DateTime
- text: string
- type: string
SubmissionController
+ createSubmission(projectID, coordID, sectionID): Submission
+ uploadDocument(sectionID, documentData): Document
+ submitForReview(submissionID)
manage
manages
handles
SubmissionCoord
ProjectCoordinator
- registrationNum: string
- firmName: string
creates
ProjectProfessional
- professional_num: string
QualifiedPerson
- role: string
ProjectInvolvement
-role: string
-status: string
DocumentController
+uploadDocument(submissionID, sectionID, file): Document
+downloadDocument(documentID): File
+deleteDocument(documentID): bool
Document
- documentID: string
- readonly: bool
- name: string
- type: string
- uploadTime: DateTime
part of
User
- nationalID: string
- name: string
- email: string
- password: string
- status: string
AuthController
+ login(email, password): bool
authenticates
ProjectUser
UserController
+registerUser(userData): User
+updateProfile(userID, updatedData): User
+getUserDetails(userID): User
+assignRole(userID, roleType): bool
AgencyController
+registerAgency(name): Agency
+assignDepartmentHead(agencyID, headID): bool
+addProjectType(typeName): ProjectType
Developer
Staff
- phonenum: number
Organization
- role_type: string
- brn: number
DepartmentHead
ProcessingOfficer
gives
head of
has
expert in
Agency
- name: string
ProjectType
- type: string
ProcessingOfficerController
+assignToSubmission(submissionID, officerID): bool
+reviewSection(sectionID, status): void
+giveWrittenDirection(submissionID, text, type): WrittenDirection

UML


status
involves
role
nationalid
email
User
name
status
ProjectUser
IsA
IsA
professionalnum
ProjectProfessional
Developer
Staff
phonenum
role_type
employed
IsA
IsA
upload by
Business Registration Number
Organization
name
ProjectCoordinator
QualifiedPerson
DepartmentHead
ProcessingOfficer
expert_of
status
creates
supervise
assigned_to
prepare
IsA
value
projectid
address
buildingwork
Project
title
type
gfa
gpr
SubmissionCoord
Section
sectionid
head_of
has
ProjectType
type
assign_to
Agency
manages
status
has
belong
submissionid
name
readonly
category
status
Submission
version
Document
documentid
uploadtime
type
name
createtime
gives
contains
precedes
type
succeeds
is_version_of
targets
paymentAmount
paid
provides
type
WrittenDirection
wdid
timestamp
text

ERD

Data Flow Architecture



Key API Endpoints Implemented

POST	/auth/login	JWT authentication — returns access token
POST	/users/register	Register new user (Coordinator / QP / Developer)
POST	/projects/	Create a new construction project
GET	/projects/{id}	Retrieve project details and submission status
POST	/submissions/{id}/upload	Upload technical documents (PDF, 3D model)
PATCH	/sections/{id}/status	Agency officer updates section review status

Implementation

Project File Structure

```
cs542gp/
  └ backend/
      └ models.py        ← DB schema
      └ database.py      ← Connection
      └ api.py           ← REST routes/JWT logic
      └ seed.py          ← Seed file
  └ frontend/
      └ login.hmtl       ← Login page
      └ project.hmtl     ← Project Display page
      └ register.hmtl    ← Register account page
  └ tests/
      └ test_utils.py
```

models.py

Defines all 20+ SQLModel table classes with typed fields, FK relationships, and constraints.

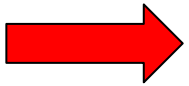
database.py

PostgreSQL async engine setup, session factory, and connection lifecycle management.

api.py

FastAPI router registration, dependency injection, CRUD endpoints for all entities, JWT token generation and verification, password hashing with bcrypt, user session handling.

Problems Encountered & Solutions

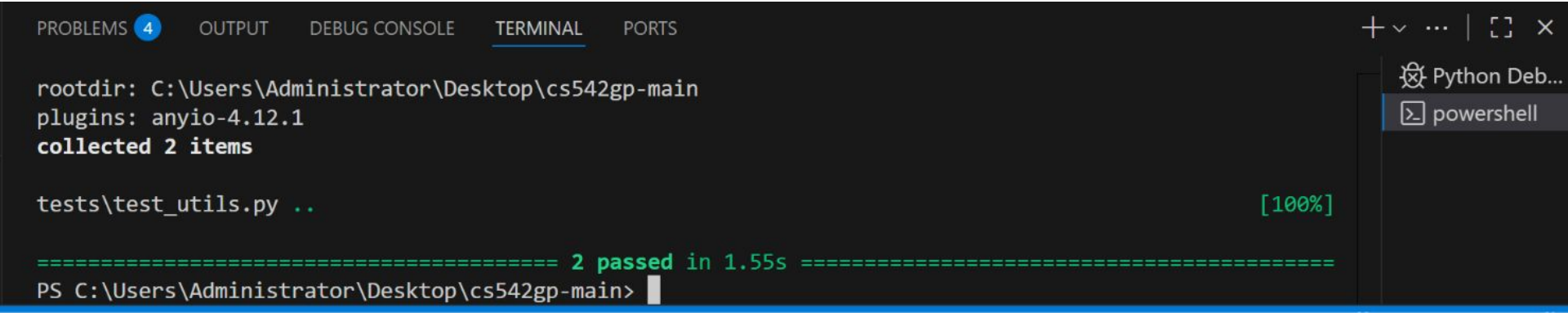
Problem1: SQLModel Inheritance Complexity		Solution: Used explicit table=True flags and separate model classes for each entity subtype to avoid ORM mapping conflicts.
Problem 2: PostgreSQL Async Connection Setup		Solution: Switched from sync to async SQLAlchemy engine; refactored session dependency injection for FastAPI lifespan events.
Problem 3: JWT Token Expiry & Refresh Logic		Solution:Implemented token expiration checks server-side with proper HTTP 401 responses; frontend redirects on expiry.
Problem 4: File Upload Size & Type Validation		Solution: Added MIME type checks and file size limits in FastAPI middleware before storage to prevent invalid uploads.

Testing

API tests

Database connection tests

Password hashing tests



Test Table

ID	Test Case	Input	Expected Output	Status
1	Hash password	"123456"	Hashed string (SHA-256)	Done
2	Verify password – correct	correct pw + hash	True	Done
3	Verify password – wrong	wrong pw + hash	False	Done
4	API root (GET /)	HTTP GET to /	200 OK, body constraints “live”	Done
5	Database connection	engine.connect()	Connection object not None	Stub
6	Get all users (GET /users/)	HTTP GET to /users/	200 OK	Done

Validation & Testing

Core Feature Status

Database Schema (20+ tables)	✓ Done
JWT Authentication & Password Hashing	✓ Done
FastAPI Backend & REST Routes	✓ Done
File Upload / Download API	○ Planned
Frontend UI (Login, Dashboard)	○ in Progress
Agency Review Workflow	○ Planned
End-to-End Integration Tests	○ Planned

Unit Test Cases

ID	Test Case	Input	Status
1	Hash password	"123456"	Pass
2	Verify correct pw	correct pw	Pass
3	Verify wrong pw	wrong pw	Pass
4	API root endpoint	GET /	Pass
5	Get all users	GET /users/	Pass
6	DB connection	engine.connect()	Stub

Overall Progress: ~50% complete — Database schema done, backend partially functional, frontend and integration tests remaining.

Lesson Learned

Database Normalization

Designing 20+ normalized tables from the start prevented data redundancy but required careful planning of FK relationships and inheritance hierarchies.

FastAPI + SQLModel Integration

Learned to leverage Pydantic type hints through SQLModel for both DB models and API schemas simultaneously, reducing boilerplate significantly.

JWT Authentication Flow

Implementing stateless JWT auth taught the team about token lifecycle, refresh strategies, and secure password hashing with bcrypt.

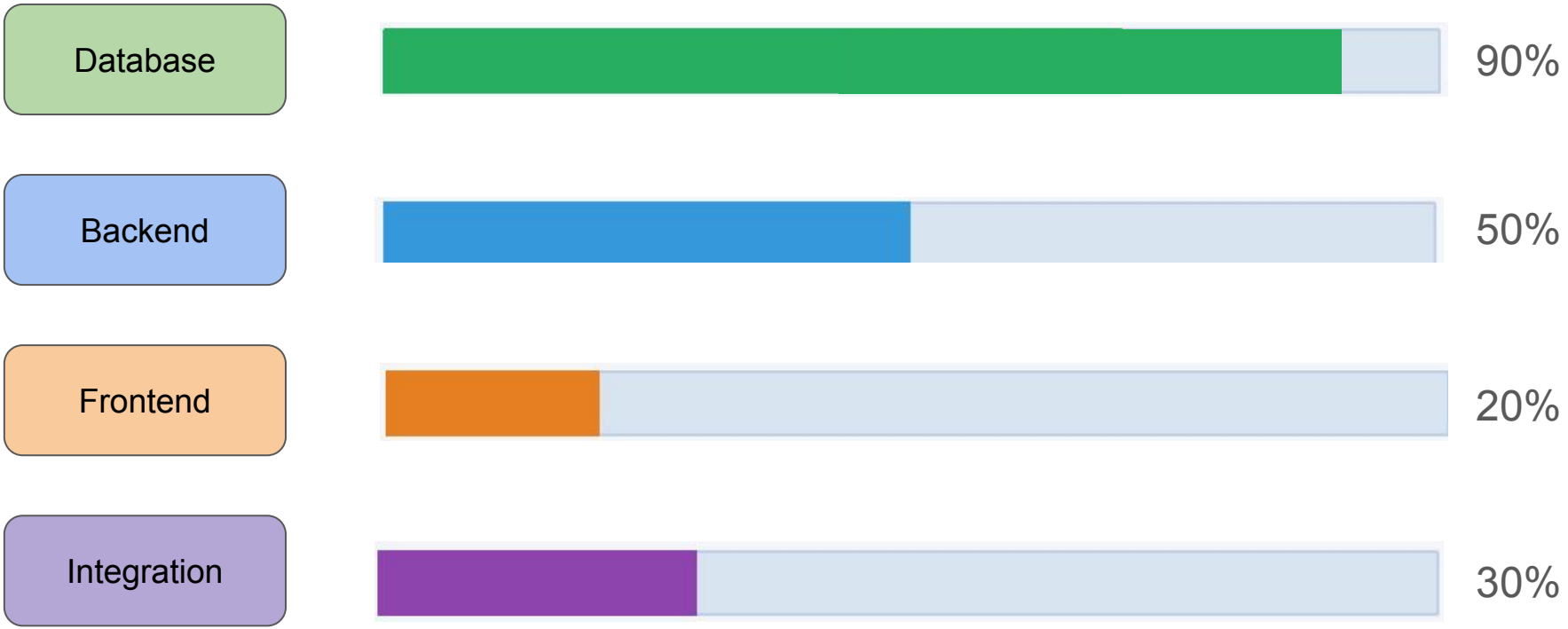
Async PostgreSQL Sessions

Transitioning to async database sessions required refactoring dependency injection patterns and understanding FastAPI lifespan context managers.

Git Collaboration & Branching

Established feature-branch workflows with pull request reviews to coordinate across 4 team members without merge conflicts.

Implementation Progress



Plan & Schedule

Background Material & References

FastAPI (Ramírez): REST API framework — JWT auth, file handling, OpenAPI docs.

PostgreSQL Docs: Core RDBMS — constraints, triggers, normalization best practices

Kim, K. (2023): FastAPI + PostgreSQL integration guide from Medium.

Ataide et al. (2023): Digital transformation of building permits — Buildings journal.

McKinsey (2016, 2017): Construction digitization gap; 14-15% productivity opportunity

SQLModel (Ramírez): ORM bridging FastAPI and PostgreSQL with Python type hints.

Tailwind CSS: Utility-first CSS for rapid, responsive frontend styling.

Ulili, S. (2025): File upload handling through FastAPI — security & multipart

NAHB (2021): Regulatory cost impact: 23.8% of new home price

Pablo (2025): Integrating 3D model viewers in browser with HTML/CSS